

Shiv Nadar University Chennai
CS3807 – Deep Learning Laboratory

Experiment 5

**Comprehensive Study of CNN Training, Regularization,
Optimization, Hyperparameter Tuning, Transfer Learning
and Cross-Validation**

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

August 27, 2026

1 Objective

The objective of this experiment was to systematically study the effect of weight initialization, regularization, optimization algorithms, CNN hyperparameters, transfer learning, fine-tuning and cross-validation on image classification performance.

The experiment used a single CNN architecture, MobileNetV2, and the Oxford-IIIT Pet dataset. The effect of individual design choices was analyzed using training/validation curves, and a suitable configuration was finally selected using 5-fold cross-validation.

2 Dataset and Experimental Setup

Dataset: Oxford-IIIT Pet Dataset.

The Oxford-IIIT Pet Dataset contains images of cats and dogs belonging to 37 breeds. The images are RGB and have different spatial dimensions. For this experiment, all images were resized to $224 \times 224 \times 3$.

Experimental considerations followed:

- RGB images were resized to 224×224 .
- Input images were normalized according to the MobileNetV2 pretrained-model requirements.
- Separate training, validation and test data were maintained.
- The test set remained untouched during hyperparameter selection.
- MobileNetV2 was used because it is computationally lighter than many large CNN architectures and is more suitable for CPU-based laboratory work.

Table 1: Actual Dataset Split Used

Split	Number of Images
Training	2944
Validation	736
Test	3669

3 MobileNetV2 Architecture

MobileNetV2 is a lightweight CNN architecture designed to reduce computational complexity while maintaining good image representation.

Its important components include:

- depthwise convolution;

- pointwise 1×1 convolution;
- inverted residual blocks;
- linear bottlenecks;
- batch normalization; and
- ReLU6 activation.

A simplified representation is:

Input ($224 \times 224 \times 3$) $\rightarrow$ Convolution / Depthwise Convolution $\rightarrow$ Pointwise
Convolution $\rightarrow$ Residual Block $\rightarrow$ Global Average Pooling $\rightarrow$ Dense $\rightarrow$ Softmax

Note: The above diagram is a simplified conceptual representation rather than the complete MobileNetV2 architecture.

4 Weight Initialization

Weight initialization determines the initial values of trainable weights before training begins. A suitable initialization can improve convergence and reduce problems such as vanishing or exploding activations.

The following initialization strategies were investigated:

- Zero initialization
- Random initialization
- Xavier/Glorot initialization
- He initialization

Results

Table 2: Weight Initialization – Summary of Results (ranked by best validation accuracy)

Initializer	Final Train Loss	Final Train Acc.	Best Val. Accuracy (%)
Xavier/Glorot	0.0071	1.0000	91.58
He	0.0065	1.0000	91.30
Random	0.0123	1.0000	90.90
Zero	3.6104	0.0279	1.77

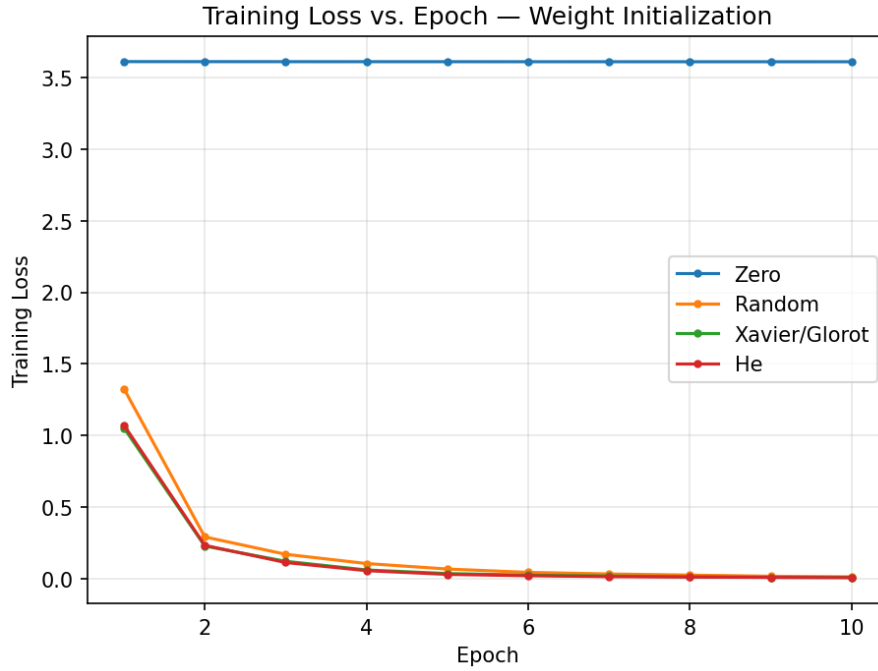


Figure 1: Training Loss vs. Epoch for Different Weight Initialization Strategies

Inference:

- The plot compares the training-loss behavior of Zero, Random, Xavier/Glorot and He initialization over 10 epochs.
- Xavier/Glorot and He initialization reduced training loss the fastest, converging to a loss near 0.007 within a few epochs.
- Random initialization converged slightly more slowly but still reached a comparably low final loss.
- Zero initialization remained flat at a training loss of approximately 3.61 for all 10 epochs, showing no meaningful learning.
- This flat behavior is consistent with the theoretical symmetry problem of zero initialization, in which all neurons in a layer receive identical gradients and therefore never differentiate from one another.

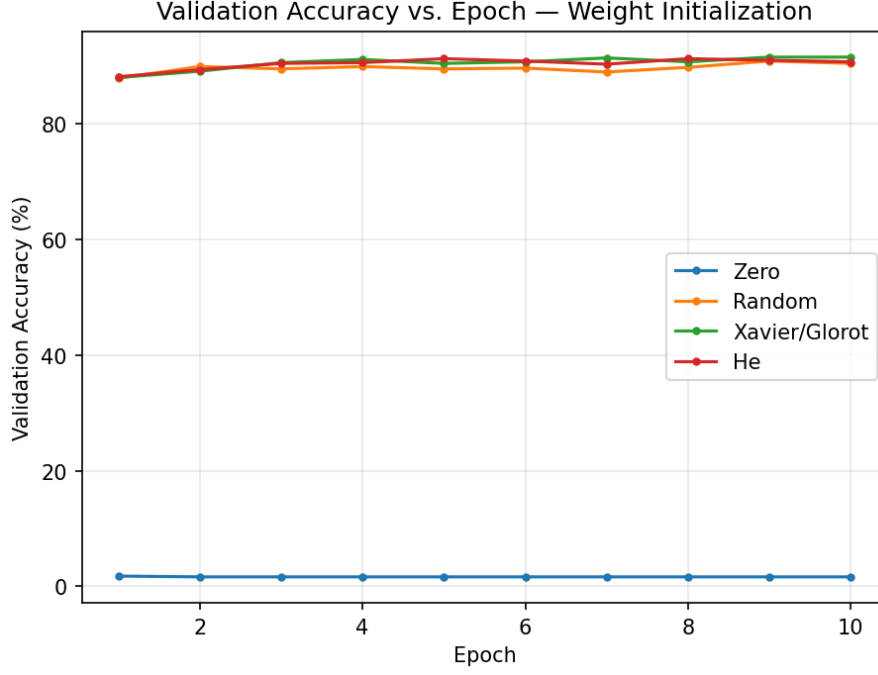


Figure 2: Validation Accuracy vs. Epoch for Different Weight Initialization Strategies

Inference:

- The plot compares validation accuracy across the four initialization strategies over 10 epochs.
- Xavier/Glorot achieved the highest observed validation accuracy (91.58%), closely followed by He (91.30%) and Random (90.90%).
- Zero initialization plateaued at approximately 1.77% validation accuracy, close to random-guess performance for a 37-class problem.
- The clear separation between Zero initialization and the other three methods indicates that a properly scaled initial weight distribution was necessary for the classifier head to learn from the frozen MobileNetV2 features in this run.

5 Regularization and Overfitting

Overfitting occurs when a model performs very well on the training data but generalizes poorly to unseen data.

The following configurations were compared:

- No regularization
- L2 regularization
- Dropout
- Batch Normalization

Table 3: Regularization – Final Epoch Metrics and Generalization Gap

Configuration	Train Acc.	Val Acc.	Gap (Train–Val)	Train Loss	Val Loss
No Regularization	1.0000	0.9103	0.0897	0.0074	0.2702
L2 Regularization	1.0000	0.9103	0.0897	0.2038	0.4348
Dropout	0.9630	0.9063	0.0567	0.1273	0.2521
Batch Normalization	0.9993	0.9063	0.0931	0.0070	0.3140

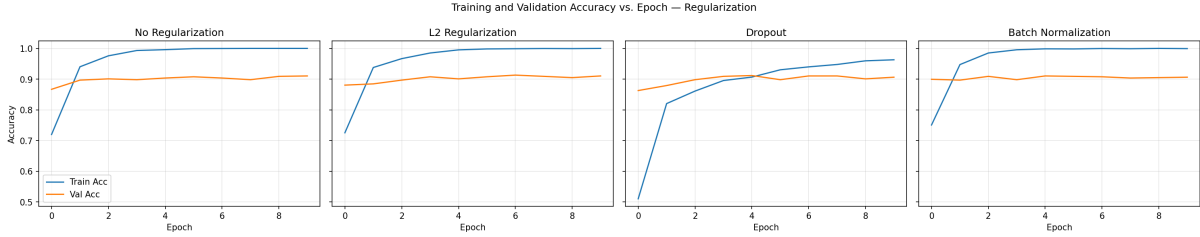


Figure 3: Training and Validation Accuracy vs. Epoch for Different Regularization Strategies

Inference:

- Training accuracy reached 1.0 for the No Regularization and L2 Regularization configurations, while validation accuracy plateaued near 0.91 for others.
- Dropout produced the smallest generalization gap of the four configurations (0.0567), with training accuracy not reaching 1.0 within the 10 epochs observed.
- Batch Normalization reached a near-perfect training accuracy (0.9993) with a validation accuracy comparable to the other configurations, producing the largest observed gap (0.0931) in this run.

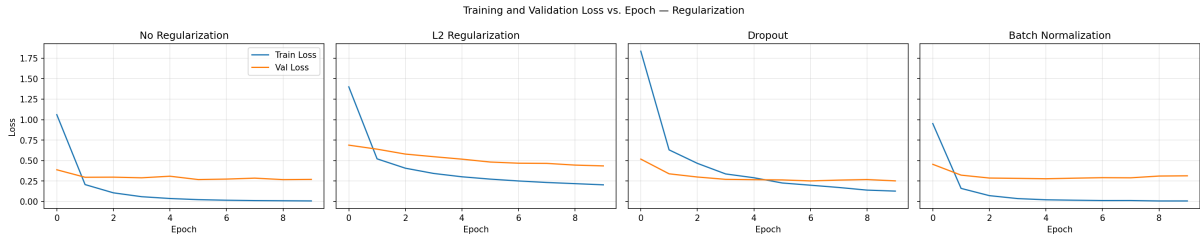


Figure 4: Training and Validation Loss vs. Epoch for Different Regularization Strategies

Inference:

- Validation loss did not show a sustained increase for any of the four configurations within the 10 epochs recorded, although training loss diverged well below validation loss in all cases except Dropout.
- L2 Regularization produced a visibly higher training loss (0.2038) than the other configurations, which is expected because the L2 penalty term is added directly to the loss being minimized.
- Dropout showed the closest tracking between training and validation loss curves among the four configurations, consistent with its smaller generalization gap in Table 3.

6 Batch Normalization

With Batch Normalization vs. Without Batch Normalization

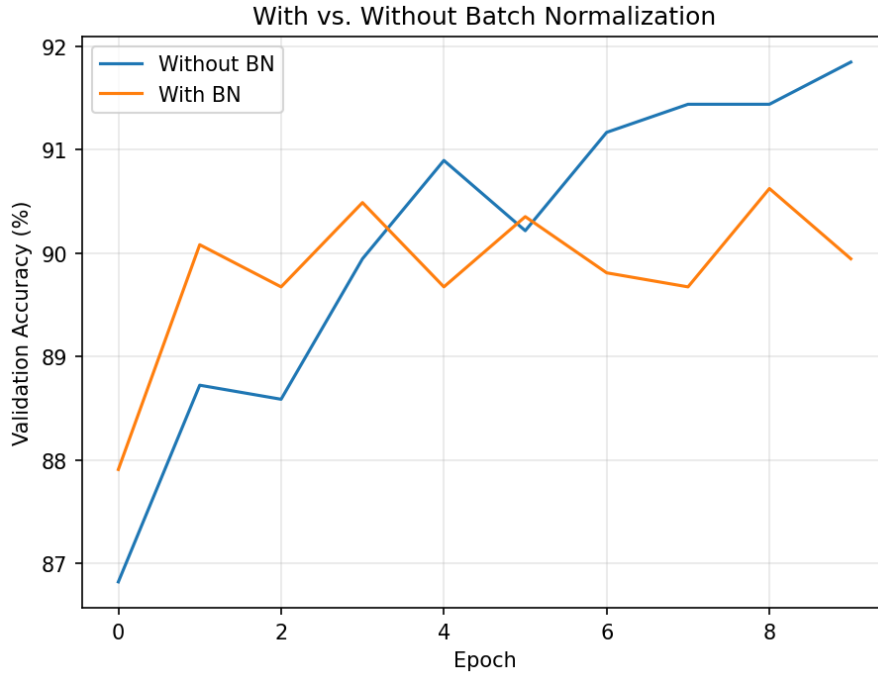


Figure 5: Validation Accuracy vs. Epoch – With BN vs. Without BN

Inference:

- The plot compares validation accuracy across 10 epochs for a classifier head trained with and without a Batch Normalization layer.
- The Without-BN configuration started lower (86.8%) but ended the highest of the two curves at epoch 10 (91.85%).
- The With-BN configuration rose faster in the first two epochs (reaching 90.1% by epoch 2) but plateaued and fluctuated between roughly 89.7% and 90.6% for the remaining epochs.
- In this particular run, adding Batch Normalization to a single-layer classifier head on top of a frozen, already-normalized MobileNetV2 feature vector did not yield a sustained accuracy improvement over the un-normalized head; both curves should be interpreted together with the regularization results in Table 3, where Batch Normalization also showed the largest generalization gap.

7 Optimization Algorithms

The following optimization methods were compared: SGD, Momentum, RMSProp and Adam. The objective was to study convergence speed, stability and final validation performance.

Table 4: Optimizer Comparison

Optimizer	Final Loss	Best Val. Accuracy	Epoch to Converge	Time (s)
SGD	2.3140	0.5109	10	99.82
Momentum	0.2469	0.9117	6	105.93
RMSProp	0.0029	0.9144	5	108.77
Adam	0.0070	0.9158	4	102.73

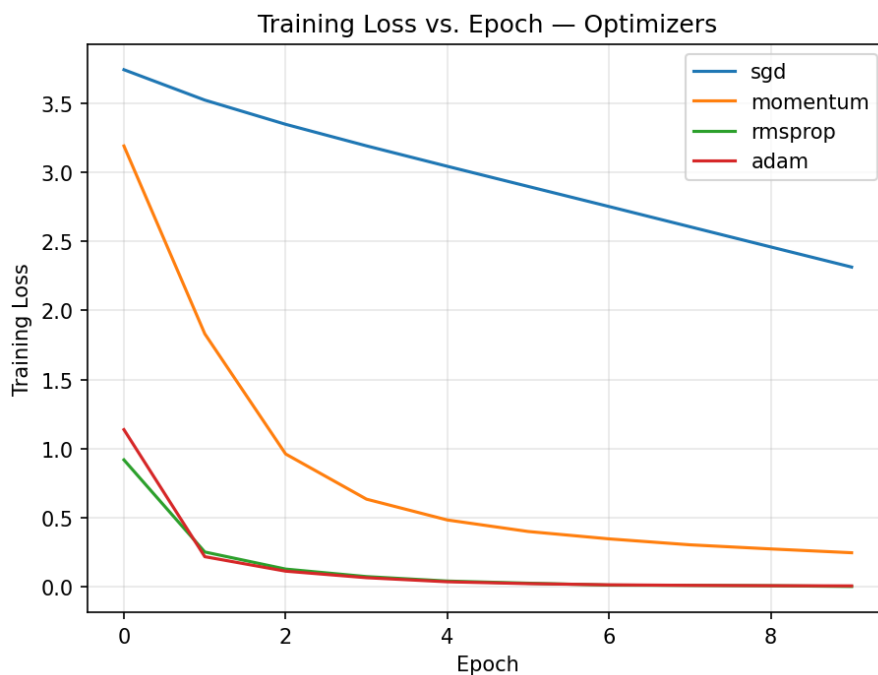


Figure 6: Training Loss vs. Epoch for Different Optimizers

Inference:

- The plot shows training loss over 10 epochs for SGD, Momentum, RMSProp and Adam.
- Plain SGD reduced loss the slowest of the four optimizers, ending at a final loss of 2.3140, far higher than the other three.
- RMSProp and Adam reduced training loss to below 0.01 within the observed epochs, the fastest of the four.
- The slow reduction under plain SGD is consistent with its use of a single fixed per-parameter learning rate without momentum or adaptive scaling, which is known to converge more slowly on this type of problem than adaptive methods.

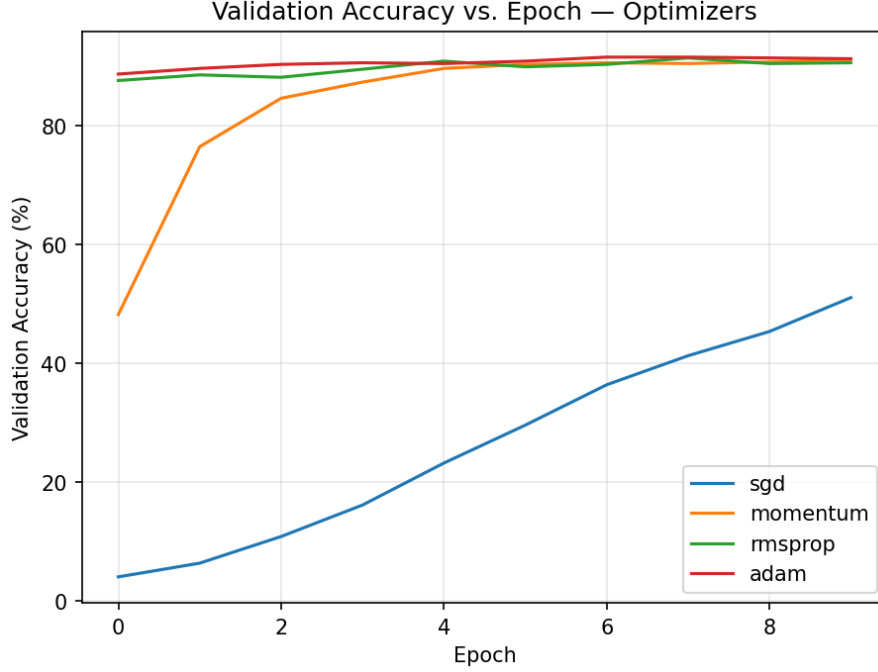


Figure 7: Validation Accuracy vs. Epoch for Different Optimizers

Inference:

- Adam reached the highest best validation accuracy observed (91.58%), followed closely by RMSProp (91.44%) and Momentum (91.17%).
- Plain SGD reached only 51.09% best validation accuracy within the 10 epochs trained, substantially below the other three optimizers.
- Adam also converged in the fewest epochs (4) according to Table 4, while SGD had not converged within the full 10 epochs recorded.
- As shown in Table 4, wall-clock training time was broadly similar across the four optimizers in this run (approximately 100–109 seconds), so the choice of optimizer did not materially change computational cost even though it changed convergence speed and final accuracy substantially.

8 CNN Hyperparameter Tuning

The following hyperparameters were investigated using a controlled one-variable-at-a-time methodology, in which a single hyperparameter was varied while the remaining settings were kept fixed at baseline values (learning rate = 0.001, batch size = 32, dropout = 0, optimizer = Adam):

Table 5: Hyperparameter Candidate Values

Hyperparameter	Suggested Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Dropout Rate	0, 0.25, 0.5
Optimizer	SGD, Adam
Fine-Tuning Learning Rate	10^{-4} , 10^{-5}
Frozen Layers	Frozen base / Partial unfreezing

Table 6: Hyperparameter Sweep – Best Validation Accuracy

Setting	Best Validation Accuracy (%)
Learning Rate = 0.001	91.30
Learning Rate = 0.0001	91.03
Batch Size = 16	91.85
Batch Size = 32	92.39
Batch Size = 64	91.30
Dropout Rate = 0.00	90.90
Dropout Rate = 0.25	92.26
Dropout Rate = 0.50	91.71

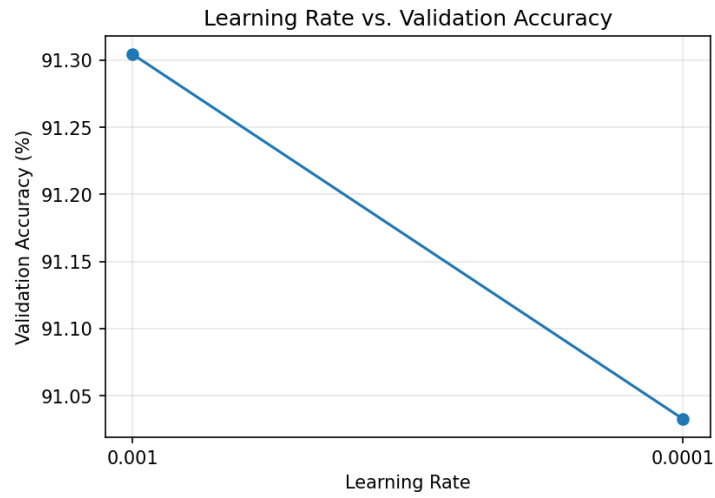


Figure 8: Learning Rate vs. Validation Accuracy

Inference:

- The plot compares best validation accuracy for learning rates 0.001 and 0.0001, with all other hyperparameters fixed at baseline.
- The higher learning rate (0.001) produced a marginally higher best validation accuracy (91.30%) than the lower learning rate (91.03%).
- The small margin between the two settings suggests that, for this frozen-backbone classifier head, validation accuracy was not highly sensitive to this particular change in learning rate within the epoch budget used.

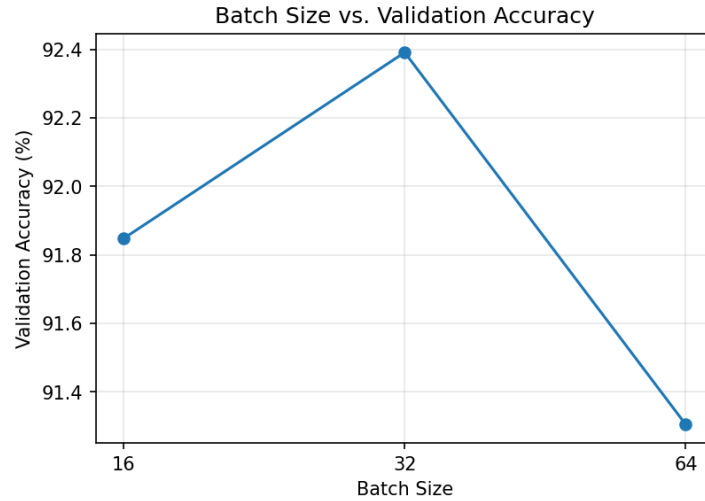


Figure 9: Batch Size vs. Validation Accuracy

Inference:

- The plot compares best validation accuracy for batch sizes 16, 32 and 64.
- Batch size 32 produced the highest best validation accuracy observed (92.39%), higher than both batch size 16 (91.85%) and batch size 64 (91.30%).
- The lower accuracy at batch size 64 relative to 32 may reflect the reduced number of gradient updates per epoch at larger batch sizes, given the fixed epoch budget used in this sweep.

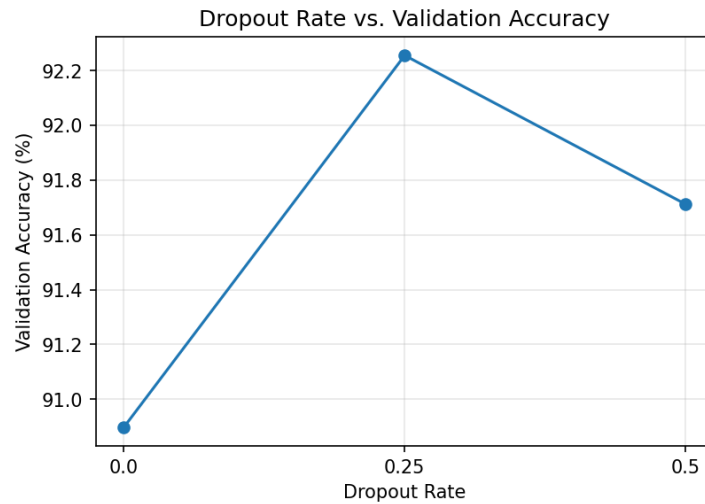


Figure 10: Dropout Rate vs. Validation Accuracy

Inference:

- The plot compares best validation accuracy for dropout rates 0.0, 0.25 and 0.5.
- A dropout rate of 0.25 produced the highest best validation accuracy observed (92.26%), ahead of dropout 0.5 (91.71%) and no dropout (90.90%).

- This non-monotonic pattern – moderate dropout outperforming both no dropout and heavy dropout – is consistent with dropout acting as a regularizer that helps most at a moderate strength, with excessive dropout removing too much useful signal within a fixed epoch budget.

9 Transfer Learning and Fine-Tuning

MobileNetV2 pretrained on ImageNet was used as the CNN backbone for all experiments in this report.

Case A: Feature Extraction

Pretrained MobileNetV2 → Freeze Base → New Classifier.

The pretrained convolutional layers were frozen and only the newly added classification layers were trained.

Case B: Fine-Tuning

Pretrained MobileNetV2 → Unfreeze Selected Layers → Small Learning Rate.

The upper layers of the pretrained network (approximately the final third of the backbone) were unfrozen and trained together with the classifier head, using a smaller learning rate (1×10^{-4}) than the feature-extraction stage.

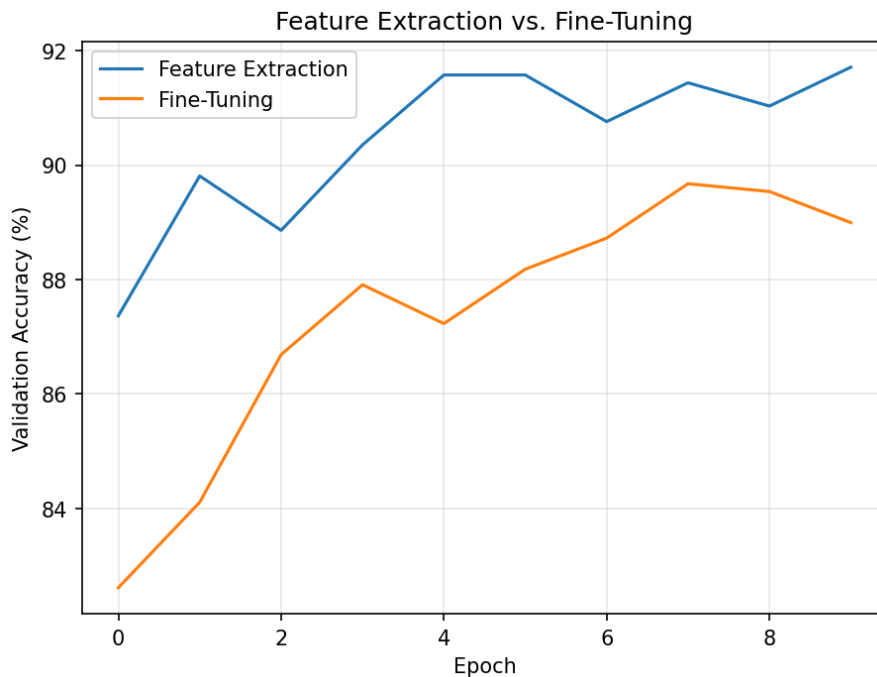


Figure 11: Feature Extraction vs. Fine-Tuning – Validation Accuracy

Inference:

- The plot compares validation accuracy over 10 epochs for Feature Extraction and Fine-Tuning.
- Feature Extraction reached a higher validation accuracy throughout training, ending at approximately 91.7% by epoch 10, compared with approximately 89.0% for Fine-Tuning.

- Fine-Tuning started from a lower validation accuracy (approximately 82.6% at epoch 1) and rose steadily but had not caught up to Feature Extraction within the 10 epochs trained.
- This gap is consistent with Fine-Tuning needing more epochs to adjust the newly unfrozen pretrained weights, since those weights start from ImageNet-trained values rather than the already-converged frozen-backbone representation used by Feature Extraction.

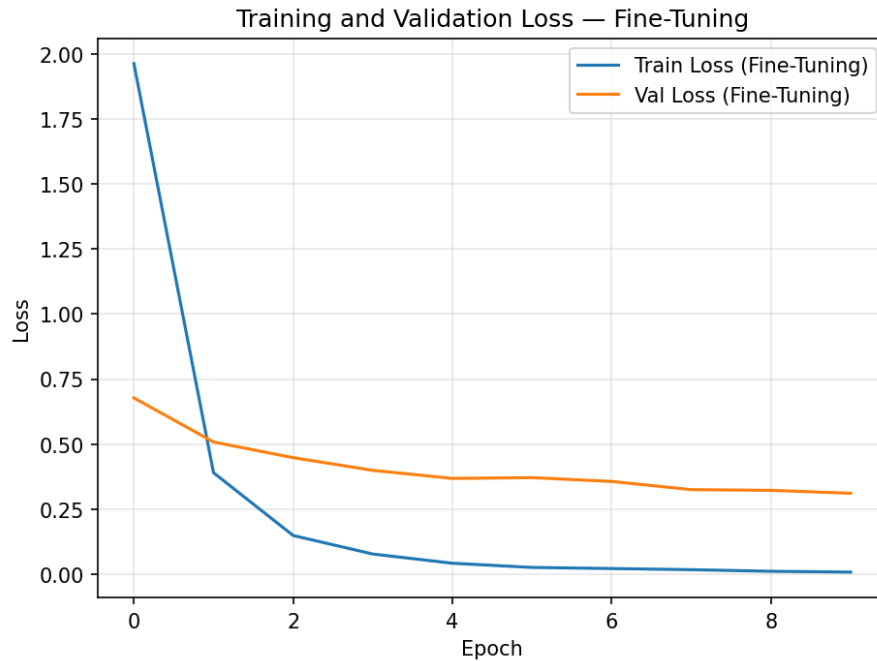


Figure 12: Training and Validation Loss – Fine-Tuning Stage

Inference:

- The plot shows training and validation loss for the Fine-Tuning configuration over its 10 training epochs.
- Both curves decreased over the epochs shown, without a sustained divergence between training and validation loss within this run.
- The continued decrease in validation loss suggests that Fine-Tuning had not yet reached its performance ceiling within the epoch budget used here, unlike Feature Extraction, which had already plateaued (Figure 11).

Discussion: Why Fine-Tuning Uses a Smaller Learning Rate

- The unfrozen layers in fine-tuning already contain useful ImageNet-pretrained weights, unlike a newly initialized classifier head.
- A small learning rate limits how much these pretrained weights are perturbed per update, reducing the risk of catastrophically overwriting useful pretrained features.
- A large learning rate applied to pretrained weights can cause large, destructive gradient updates early in training, effectively discarding the benefit of transfer learning.

10 K-Fold Cross-Validation

Four candidate configurations, selected from the preceding individual studies, were evaluated using 5-fold cross-validation on the training data. The independent test set was not used during this stage.

For $K = 5$,

$$\bar{A} = \frac{1}{5} \sum_{i=1}^5 A_i, \quad SD = \sqrt{\frac{1}{5} \sum_{i=1}^5 (A_i - \bar{A})^2}.$$

The result is reported as $\bar{A} \pm SD$.

Table 7: 5-Fold Cross-Validation Results

Configuration	F1	F2	F3	F4	F5	Mean	SD
C1_baseline_adam	0.9253	0.9063	0.9103	0.9035	0.9076	0.9106	0.0077
C2_dropout_adam	0.9171	0.9049	0.9185	0.8927	0.9130	0.9092	0.0095
C3_bn_adam	0.9063	0.9090	0.9130	0.9076	0.9158	0.9103	0.0035
C4_l2_rmsprop	0.9076	0.9022	0.9198	0.8954	0.8967	0.9043	0.0089

where C1 = baseline (no regularization, Adam), C2 = Dropout (0.5, Adam), C3 = Batch Normalization (Adam), C4 = L2 regularization with RMSProp.

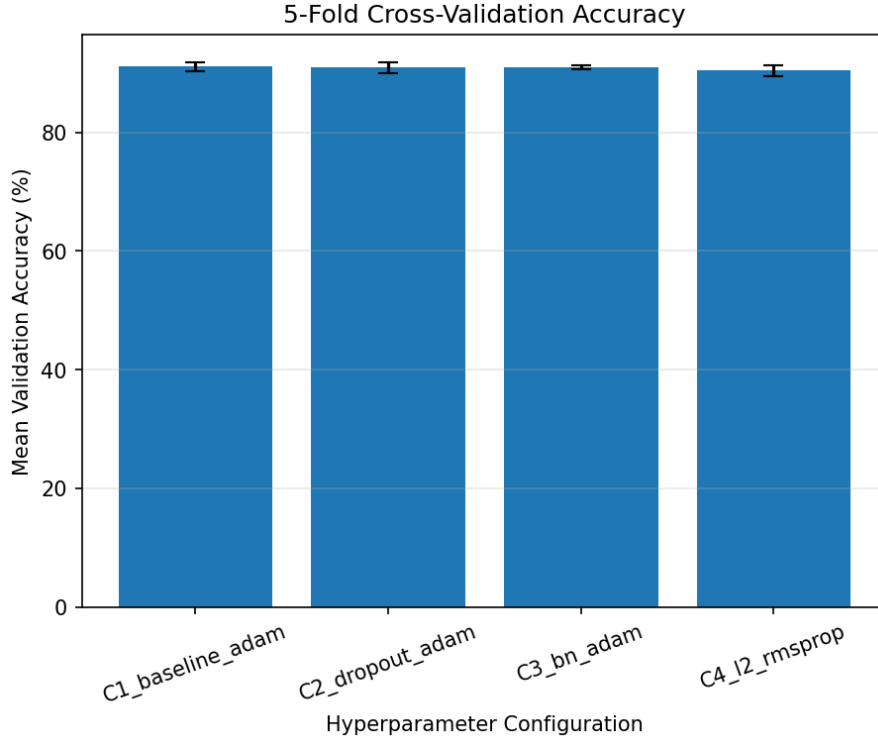


Figure 13: 5-Fold Cross-Validation Accuracy with Standard-Deviation Error Bars

Inference:

- The plot shows mean 5-fold cross-validation accuracy for the four candidate configurations, with error bars showing one standard deviation.
- C1_baseline_adam obtained the highest mean accuracy (0.9106), closely followed by C3_bn_adam (0.9103) and C2_dropout_adam (0.9092); C4_l2_rmsprop was lowest (0.9043).

- C3_bn_adam had the smallest standard deviation of the four configurations (0.0035), indicating the most consistent performance across folds, even though its mean accuracy was marginally below C1.
- Because C1_baseline_adam had both the highest mean accuracy and a competitively low standard deviation (0.0077), it was selected as the configuration for final model evaluation.

11 Final Model Evaluation

The C1_baseline_adam configuration, selected by 5-fold cross-validation, was retrained using the complete training data. The independent test set was kept untouched until this stage and was then used for a single final evaluation.

Table 8: Final Model Evaluation	
Metric	Value
Mean CV Accuracy	0.9106
CV Standard Deviation	0.0077
Test Accuracy	0.8948
Precision (macro)	0.8955
Recall (macro)	0.8940
F1-score (macro)	0.8936
Training Time (s)	105.49
Number of Parameters	2,426,725

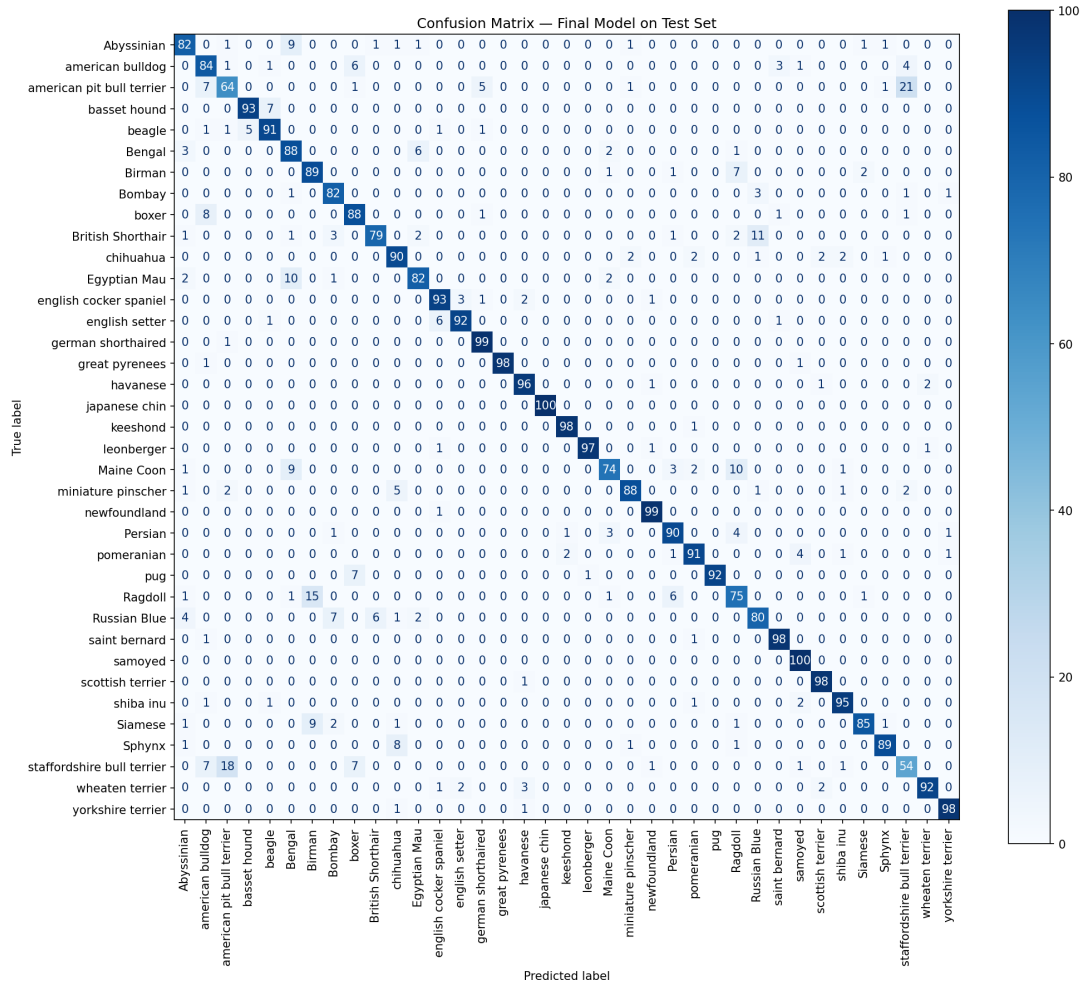


Figure 14: Confusion Matrix – Final Model on Test Set

Inference:

- The best-classified classes, each with 98–100 correct predictions out of their test samples in the confusion matrix, included Japanese Chin (100), Samoyed (100), German Shorthaired (99), Newfoundland (99), Great Pyrenees (98), Keeshond (98), Saint Bernard (98), Scottish Terrier (98) and Yorkshire Terrier (98).
- The most frequently confused pair was Staffordshire Bull Terrier and American Bulldog: 18 Staffordshire Bull Terrier test images were misclassified as American Bulldog, and the Staffordshire Bull Terrier class had the lowest diagonal value observed (54).
- American Pit Bull Terrier was also frequently confused with Staffordshire Bull Terrier (21 images), and Ragdoll was confused with Birman (15 images).
- A possible visual reason for these specific confusions is that Staffordshire Bull Terrier, American Pit Bull Terrier and American Bulldog are all short-haired, muscular dog breeds with broadly similar head and body shapes, and Ragdoll and Birman are both long-haired, pale-coated cat breeds – these are exactly the kinds of fine-grained, visually-similar breed pairs the Oxford-IIIT Pet dataset is known to be difficult on.

12 Overall Results

Table 9: Overall Results Across Experiment Stages

Configuration	Best Validation Accuracy (%)	Test Accuracy (%)
Best Initialization	91.58	—
Best Regularization	91.30	—
Best Optimizer	91.58	—
Best Hyperparameters	92.39	—
Fine-Tuned Model	89.67	—
Final Selected Model	91.06 (CV Mean)	89.48

- The Best Hyperparameters configuration (batch size 32, dropout 0.25) achieved the highest single-stage validation accuracy recorded (0.9239) among all individual studies.
- The Fine-Tuned Model recorded a lower validation accuracy (0.8967) than Feature Extraction within the epoch budget used, consistent with the observation in Figure 11 that Fine-Tuning had not yet converged.
- The final selected model’s test accuracy (0.8948) was lower than its cross-validation mean accuracy (0.9106); this is an expected pattern, since cross-validation accuracy is measured on validation folds drawn from the same distribution as training data with model selection influence, while test accuracy reflects a single evaluation on entirely unseen data.

13 Discussion Questions

1. Difference Between Parameters and Hyperparameters

Parameters are learned during training, while hyperparameters such as learning rate, batch size and dropout are set before training.

2. Why Is Weight Initialization Important?

Weight initialization provides a good starting point for training and affects how well and how quickly the model learns.

3. Why Can Zero Initialization Be Problematic for Neural Networks?

Zero initialization makes neurons learn the same features, which prevents proper learning and caused only 1.77% validation accuracy in our experiment.

4. Compare Xavier and He Initialization

Xavier is generally suited to sigmoid/tanh activations, while He initialization is designed mainly for ReLU-based networks.

5. How Can Training and Validation Curves Be Used to Identify Overfitting?

Overfitting occurs when training performance improves while validation performance stops improving or becomes worse.

6. How Does Dropout Reduce Overfitting?

Dropout randomly disables neurons during training, making the model less dependent on specific neurons and reducing overfitting.

7. What Is the Purpose of Batch Normalization?

Batch Normalization normalizes activations during training, which can make training more stable.

8. Explain the Numerical Batch Normalization Example

For $x = [2, 4, 6, 8]$, the mean is 5 and variance is 5, giving normalized values of approximately $[-1.342, -0.447, 0.447, 1.342]$.

9. What Are the Roles of γ and β ?

γ scales the normalized values and β shifts them, allowing the network to learn the required scale and position.

10. Compare SGD, Momentum, RMSProp and Adam

SGD uses the current gradient, Momentum uses previous gradients, RMSProp adapts the learning rate, and Adam combines both approaches.

11. What Happens When the Learning Rate Is Too Large?

A very large learning rate can cause unstable updates and make the training loss diverge.

12. What Happens When the Learning Rate Is Too Small?

A very small learning rate makes training slow and may prevent the model from reaching good performance within the available epochs.

13. What Is the Effect of Increasing Batch Size?

Increasing batch size gives more stable gradients but fewer updates per epoch, and in our experiment batch size 32 performed best.

14. Explain Stride and Padding

Stride controls how far the filter moves, while padding adds pixels around the input to control the output size.

15. Why Is MobileNetV2 Computationally Efficient?

MobileNetV2 uses depthwise separable convolutions and inverted residual blocks to reduce computation and parameters.

16. What Is Depthwise Separable Convolution?

Depthwise separable convolution performs depthwise convolution followed by a 1×1 pointwise convolution to reduce computation.

17. What Is Transfer Learning?

Transfer learning uses knowledge learned from a pretrained model such as ImageNet-trained MobileNetV2 for a new task.

18. Differentiate Feature Extraction and Fine-Tuning

Feature extraction freezes the pretrained layers, while fine-tuning unfreezes some of them and updates their weights.

19. Why Is a Smaller Learning Rate Generally Used During Fine-Tuning?

A smaller learning rate prevents large updates from damaging the useful pretrained features.

20. Why Is K-Fold Cross-Validation Useful for Hyperparameter Selection?

K-Fold cross-validation tests a model on multiple data splits, giving a more reliable estimate of its performance.

21. Why Must the Test Set Remain Untouched During Tuning?

The test set must remain untouched so that it gives an unbiased measure of the final model's performance.

22. Why Should Mean and Standard Deviation Both Be Reported?

The mean shows the average performance, while the standard deviation shows how consistent the model is across folds.

23. Is the Highest Validation Accuracy Always Sufficient to Select a Model?

No, model selection should also consider consistency, generalization and other practical factors.

14 References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sergey Ioffe and Christian Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," ICML, 2015.
3. Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov and Liang-Chieh Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," CVPR, 2018.
4. Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman and C. V. Jawahar, "Cats and Dogs," IEEE Conference on Computer Vision and Pattern Recognition, 2012.
5. TensorFlow Documentation.
6. Keras Documentation.